Computer Organization and Architecture	
Lab #:	**11**
Lab Title:	Combinational and Sequential circuits
Submitted by:	

Names	**Registration #**
Sumaira Safeer	FA22-BCE-019

Rubrics name & number		**Marks**	
		In-Lab	**Post-Lab**
Engineering Knowledge	***R2: Use of Engineering Knowledge and follow Experiment Procedures:*** *Ability to follow experimental procedures, control variables, and record procedural steps on lab report.*		
Problem Analysis	***R6: Experimental Data Analysis:*** *Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.*		
Design	***R8: Best Coding Standards:*** *Ability to follow the coding standards and programming practices.*		
Modern Tools Usage	***R9: Understand Tools:*** *Ability to describe and explain the principles behind and applicability of engineering tools.*		
Individual and Teamwork	***R12: Individual Work Contributions:*** *Ability to carry out individual responsibilities.*		
	R13: Management of Team Work: *Ability to appreciate, understand and work with multidisciplinary team members.*		

Rubrics #	**R2**	**R6**	**R8**	**R9**	**R12**	**R13**
In Lab						
Post- Lab						

Combinational and Sequential circuits

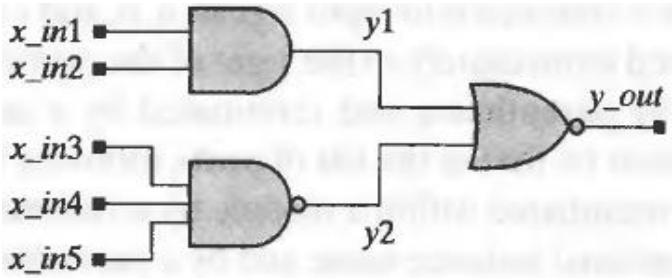
Objectives:

This lab includes a refresher of combinational and sequential circuits.

- Review/refresher of combinational circuits.
- Review/refresher of sequential circuits.

Task 1:

Five-input and-or-invert (AOI) circuit.



Verilog Code:

```
module AOI5_CA0 (y_out, x_in);
input [4:0] x_in;    // Inputs of UUT decided as ?reg?
output y_out;        //Outputs of UUT (is this case module verilog?)

assign y_out = ~((x_in[0] & x_in[1])|(x_in[2]& x_in[3]& x_in[4]));
endmodule
```

TestBench:

```
module TB_AOI5_CA0;
   reg [4:0] x_in;        // Inputs of UUT decided as ?reg?
   wire y_out;            //Outputs of UUT (is this case module verilog?) decided as // wire
```

```
AOI5_CA0 uut (                // Instantiate the Unit Under Test (UUT)
    .x_in(x_in),
    .y_out(y_out)
);
   initial begin
    // Apply test cases
    x_in = 5'b00000;
    #10;          // All inputs 0
    x_in = 5'b00011;
    #10;          // x_in[0]=1, x_in[1]=1
    x_in = 5'b11100;
    #10;          // x_in[2]=1, x_in[3]=1, x_in[4]=1
    x_in = 5'b11111;
    #10;           // All inputs 1
    x_in = 5'b10101;
    #10;           // Mixed inputs
    x_in = 5'b01010;
     #10;          // Alternate inputs
    $stop;        // End simulation
    end
   endmodule
```

Result:

Task 2:

implements a 32-bit 4-1 mux using conditional operator.

Verilog Code:

```
module mux_4_to_1 (
    input [31:0] I0,   // 32-bit input 0
    input [31:0] I1,   // 32-bit input 1
    input [31:0] I2,   // 32-bit input 2
    input [31:0] I3,   // 32-bit input 3
    input S1,          // Select line 1
    input S0,          // Select line 0
    output reg [31:0] Y // 32-bit output
    );
    always @(*) begin
    // Case structure to implement the 4-to-1 MUX
     case ({S1, S0})
     2'b00: Y = I0; // When S1=0, S0=0, output I0
      2'b01: Y = I1; // When S1=0, S0=1, output I1
      2'b10: Y = I2; // When S1=1, S0=0, output I2
```

```
2'b11: Y = I3; // When S1=1, S0=1, output I3
      default: Y = 32'b0; // Default case (safe, if needed)
      endcase
       end
    endmodule
```

TestBench:

```
module tb_mux_4_to_1;
  // Declare inputs as reg and output as wire
  reg [31:0] I0, I1, I2, I3;        // Inputs of UUT decided as ?reg?
  reg S1, S0;
  wire [31:0] Y;                    //Outputs of UUT (is this case module verilog?) //wire

  mux_4_to_1 uut (                  // Instantiate the MUX module
     .I0(I0),
     .I1(I1),
     .I2(I2),
     .I3(I3),
     .S1(S1),
     .S0(S0),
     .Y(Y)
  );
  initial begin
     // Initialize inputs
     I0 = 32'h12345678;
     I1 = 32'h23456789;
     I2 = 32'h34567890;
     I3 = 32'h45678901;

     S1 = 0; S0 = 0;                // Instantiate the MUX module     // Select I0
```

```
#10;
$display("S1=%b S0=%b, Y=%h", S1, S0, Y);
S1 = 0; S0 = 1;                    // Select I1
#10;
$display("S1=%b S0=%b, Y=%h", S1, S0, Y);

S1 = 1; S0 = 0;                    // Select I2
#10;
$display("S1=%b S0=%b, Y=%h", S1, S0, Y);

S1 = 1; S0 = 1;                    // Select I3
#10;
$display("S1=%b S0=%b, Y=%h", S1, S0, Y);
$finish;
end
endmodule
```

Result: